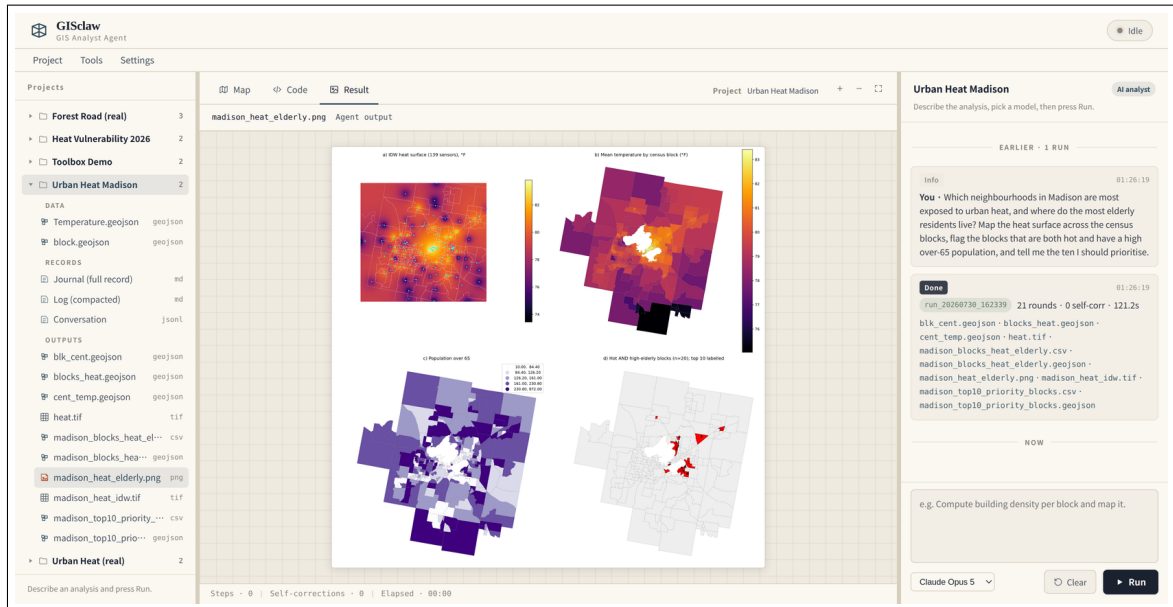


GISclaw

사용 설명서

필요한 공간 분석을 말하면, 완성된 결과를 받습니다.



GISclaw는 지리공간 분석 자동화 도구입니다. 일상적인 언어로 질문하면 워크플로를 계획하고 사용자의 데이터에 대해 실행한 뒤, 레이어와 지도, 표, 그리고 그 과정을 기록한 문서를 돌려줍니다. 본 설명서는 설치, 일상적인 사용, 남는 기록, 그리고 몇 달에 걸친 프로젝트를 계속 읽을 수 있게 하는 구조를 다룹니다.

Docker, GitHub, 모델 API를 다뤄 본 적이 없다면? 부록 A 〈처음부터 설치하기〉 (11쪽) 부터 시작하십시오. 사전 지식을 가정하지 않고 설치와 키 발급을 거쳐 첫 분석을 마칠 때까지 안내한 뒤 이곳으로 돌아오게 합니다.

Contents

1 GISclaw가 하는 일	3
1.1 일반적인 요청	3
2 설치	3
2.1 요구 사항	3
2.2 절차	3
3 일상적인 사용	4
3.1 프로젝트 생성과 데이터 첨부	4
3.2 좋은 요청 작성법	5
3.3 실행 중 확인할 것	5
4 실행을 규율하는 원칙	5
5 프로젝트에 남는 것	6
5.1 압축 로그	6
6 요청의 성격에 맞는 응답	6
7 상시 적용되는 선호	7
8 Skills	7
9 연산 도구 상자	8
10 데이터의 위치	9
11 문제 해결	9
12 연구 논문과의 관계	9
13 결과에 의존하기 전에	9
14 라이선스와 출처 표시	10
A 처음부터 설치하기	11
A.1 필요한 세 가지	11
A.2 1단계 — Docker 설치	11
A.2.1 Windows 10 또는 11	11
A.2.2 macOS	12
A.2.3 Linux(Ubuntu, Debian 계열)	12

A.2.4	제대로 설치됐는지 확인	12
A.2.5	다음에 입력하게 될 명령	12
A.3	2단계 — GISclaw 파일 받기	12
A.3.1	git 없이 받기(가장 간단)	13
A.3.2	git으로 받기	13
A.3.3	그 폴더에서 터미널 열기	13
A.4	3단계 — API 키 발급	13
A.5	4단계 — GISclaw 실행	14
A.6	5단계 — 첫 분석	14
A.7	자주 쓰게 될 명령	14
A.8	작업물이 저장되는 곳과 이후 업데이트	15
A.8.1	새 버전으로 업데이트	15
A.9	설치가 잘 되지 않을 때	16
A.10	비용을 낮게 유지하려면	16

1 GISclaw가 하는 일

공간 분석은 대개 잘 정립된 연산의 조합입니다. 재투영, 보간, 조인, 집계, 분류, 지도화. 시간이 드는 지점은 이들을 올바른 순서로 조립하는 일이며, 오류도 여기에 숨습니다.

GISclaw는 도메인 연구자가 실제로 말하는 형태의 요청을 그대로 받습니다. 표지의 4패널 그림은 다음 한 문장에서 생성되었습니다.

Which neighbourhoods in Madison are most exposed to urban heat, and where do the most elderly residents live? Map the heat surface across the census blocks, flag the blocks that are both hot and have a high over-65 population, and tell me the ten I should prioritise.

이 문장으로부터 GISclaw는 두 레이어를 확인해 좌표계가 다름을 발견하고 재투영했으며, 139개 관측값으로 IDW 열 표면을 만들고, 269개 인구조사 블록의 구역 평균을 계산하고, 온도와 65세 이상 인구밀도를 정규화해 결합·순위화하고, 그림을 그린 뒤 열 개의 파일을 저장했습니다. 총 21단계, 121초.

1.1 일반적인 요청

사용자의 요청	산출물
침수 수위 2 m 이상일 때 병원 접근이 차단되는 구간은?	침수 범위, 영향받는 도로 구간, 고립되는 구역
계획 도로 500 m 이내에 포함되는 보호림은 얼마인가?	버퍼, 교차, 보호등급별 영향 면적, 영향도
소방서 서비스가 부족한 지역은?	서비스 반경, 공백 구역, 공백별 인구
두 시점의 산림 피복을 비교하고 손실량을 산출하라.	변화 래스터, 증가·감소·순변화, 발산형 배색 지도

2 설치

2.1 요구 사항

Docker와 Docker Compose, 이미지용 디스크 약 3 GB, 그리고 하나의 모델 제공자 API 키. GIS 소프트웨어를 별도로 설치할 필요는 없습니다. 컨테이너가 오픈소스 지리공간 스택 전체를 포함합니다.

Docker, GitHub, 모델 API를 써 본 적이 없다면? 아래 네 단계는 이미 환경이 갖춰진 독자를 위한 요약입니다. 부록 A에서 같은 내용을 처음부터 다룹니다 — 운영체제별 Docker 설치, git 없이 파일 내려받기, 제공자별 키 발급, 그리고 각 단계에서 막혔을 때의 대처까지.

2.2 절차

1. `git clone https://github.com/geumjin99/GISclaw.git` 후 `cd GISclaw`
2. `docker compose up -d`
3. `http://localhost:8765` 열기
4. **Settings** → **API keys...** 에서 키를 붙여넣고 **Test**

키가 저장되는 위치. 키는 서버 측 `./projects/.gisclaw/settings.json`에 권한 600으로 저장됩니다. 이미지에 포함되지 않고 브라우저로 전달되지 않으며, 인터페이스에는 `sk-abc...wxyz` 형태의 마스크만 반환됩니다. CI나 공용 호스트에서는 `.env`의 환경 변수를 대체 수단으로 사용할 수 있습니다.

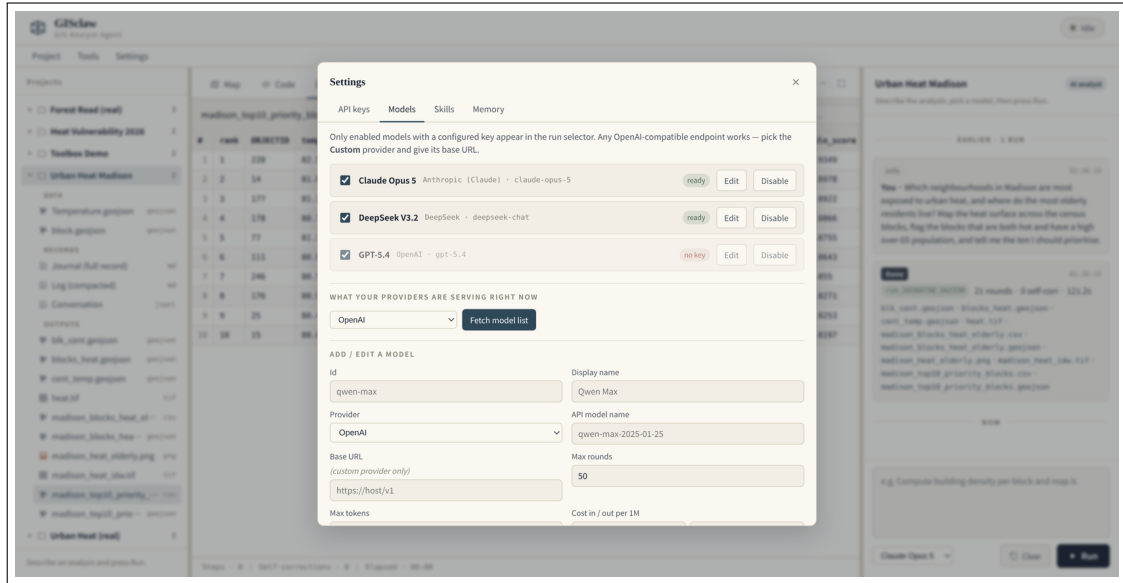


Figure 1: 모델 레지스트리. 목록은 각 제공자로부터 실시간으로 가져오므로 새로 출시된 모델도 본 소프트웨어의 갱신을 기다릴 필요가 없으며, 키가 동작하는 모델만 실행 선택기에 표시됩니다.

3 일상적인 사용

3.1 프로젝트 생성과 데이터 첨부

프로젝트는 작업 폴더입니다. **Project** → **New project...** 로 만들고, **Project** → **Add data...** 로 데이터를 첨부합니다. 이때 열리는 것은 마운트된 작업 공간을 루트로 하는 서버 측 파일 브라우저입니다. 트리에서 프로젝트 폴더를 우클릭해도 됩니다.

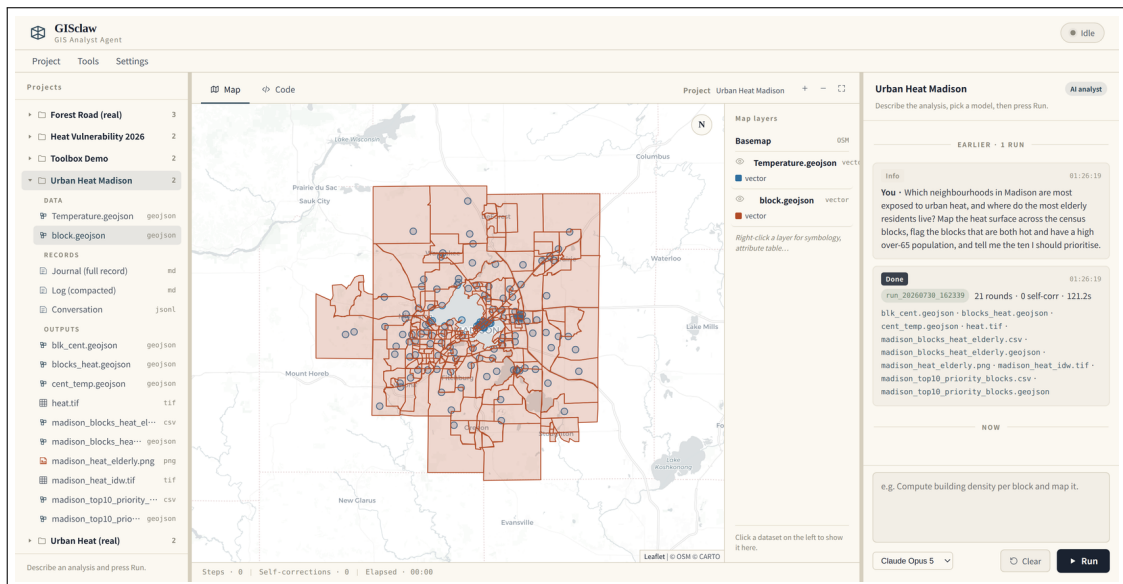


Figure 2: 지도에 렌더링된 레이어, 왼쪽의 프로젝트 트리, 오른쪽의 대화. 레이어를 우클릭하면 심볼 설정, 속성 테이블, 확대를 사용할 수 있습니다.

3.2 좋은 요청 작성법

목표를 밝히고, 중요한 제약을 명시하고, 무엇을 돌려받고 싶은지 적습니다. 에이전트가 스키마를 직접 읽으므로 컬럼명을 제공할 필요는 거의 없습니다.

약함 열 분석을 해 줘.

사용 가능 온도 지점을 보간하고 블록별로 집계해 줘.

좋은 TemperatureF 지점을 연속 표면으로 보간하고, 인구조사 블록별 구역 평균을 계산하고, blocks_meantemp.geojson 과 choropleth PNG 를 pred_results/ 에 저장하고, 가장 더운 다섯 블록과 값이 없는 블록 수를 알려 줘.

확인 가능한 숫자를 함께 요구하는 것(가장 더운 다섯 블록, 값이 없는 개수)은 완료되었지만 정확하지 않은 실행을 잡아내는 가장 저렴한 방법입니다.

3.3 실행 중 확인할 것

Thought, Action, Observation이 실시간으로 대화에 흐릅니다. 생성된 코드는 **Code** 탭에, 그림은 **Result** 에, 표와 텍스트는 **Table** 에 나타나며, 벡터·래스터 산출물은 지도에 자동으로 표시됩니다.

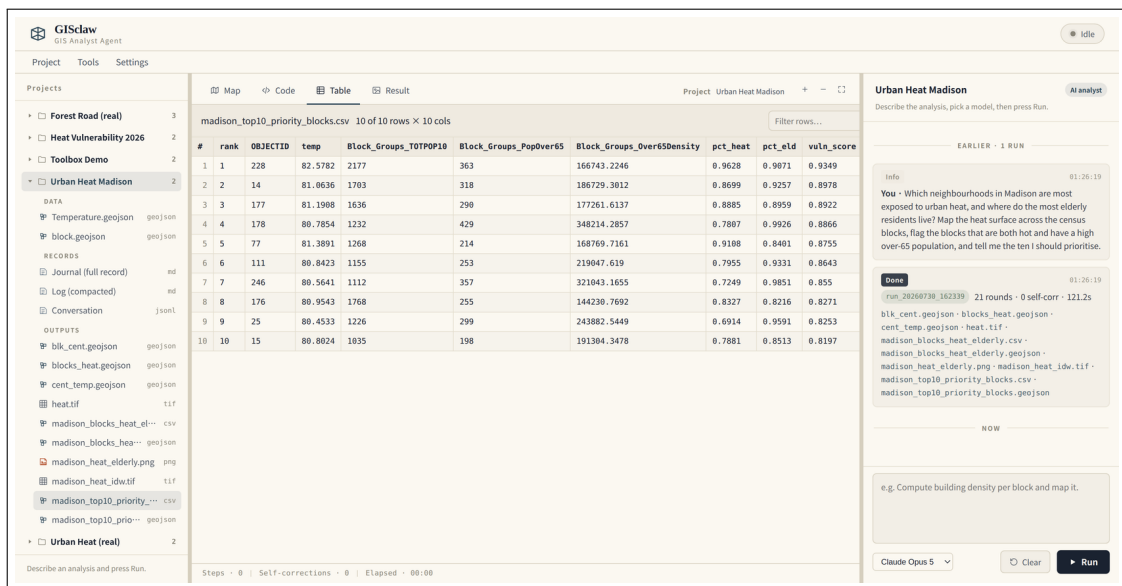


Figure 3: 요청, 산출물이 포함된 실행 요약, 그리고 뷰어에서 열린 결과 표.

4 실행을 규율하는 원칙

GISclaw는 영속적인 Python 샌드박스 위에서 ReAct 루프를 수행합니다. 데이터를 살펴보고, 다음 단계를 추론하고, 몇 줄을 작성하고, 결과를 읽고, 이어갑니다.

운영 원칙은 실제 다단계 GIS 과제에 대한 약 1,800회의 통제 실행에서 도출되었으며 모든 분석에 적용됩니다.

1. **코드보다 전체 경로를 먼저 확정한다.** 가장 흔한 실패는 잘못된 계획에 대한 올바른 코드입니다.
2. **스키마는 데이터에서 읽는다.** 컬럼명을 추측하지 않습니다.
3. **CRS를 최우선으로 다룬다.** 투영 혼재는 일상이며, 거리·면적 연산은 투영 좌표계에서 수행하고 조인 후에는 결측치를 출력합니다.

4. **결정적 연산을 우선 사용한다.** 동등한 코드를 직접 작성하기보다 먼저 사용합니다.
5. **결측값을 조용히 채우지 않는다.** 관측이 없는 단위는 NULL로 둡니다. 불가피한 경우 이유, 개수, 플래그 컬럼이 필요하며 요약에도 명시해야 합니다.
6. **재시도보다 진단한다.** 실제 오류는 트레이스백의 끝에 있습니다.
7. **종료 전 검증한다.** 값의 범위, 개수, 지도화한 필드가 실제로 변하는지.

다섯 번째 원칙이 존재하는 이유. 269개 단위 중 170개를 나머지 99개의 평균으로 채우면, 패턴의 3분의 2가 만들어진 것임에도 완결되어 보이는 지도가 나옵니다. 자동 검사도 모두 통과합니다. 99개 단위만 다루었다고 분명히 밝히는 편이 더 가치 있습니다.

5 프로젝트에 남는 것

projects/<프로젝트>/	
— data/	첨부한 데이터
— outputs/	산출물
— JOURNAL.md	전체 기록: 모든 실행, 모든 라운드
— LOG.md	압축 요약, 실행당 한 항목
— chat.jsonl	대화 기록, 화면에서 재구성
— runs/run_<타임스탬프>/	
— code.py	실제로 실행된 코드
— trace.jsonl	모든 Thought / Action / Observation
— pred_results/	해당 실행의 산출물 (덮어쓰지 않음)

새로고침하든, 컨테이너를 재시작하든, 세 달 뒤에 돌아오든 대화는 chat.jsonl에서 복원됩니다. 과거 실행을 클릭하면 추론이 재생되고 당시 실행한 코드가 불러와집니다.

어느 사본을 신뢰할 것인가. outputs/는 편의를 위한 사본이며 이후 실행이 같은 파일명으로 덮어쓸 수 있습니다. runs/.../pred_results/는 변경되지 않으므로 특정 결과는 항상 그곳에서 재현할 수 있습니다.

5.1 압축 로그

장기 프로젝트에는 아무도 다시 읽지 않는 기록이 쌓입니다. 실행이 끝날 때마다 GISclaw는 모델 호출 한 번으로 추적을 다섯 항목(Result, Method, Numbers, Caveats, Carries forward)으로 압축해 LOG.md에 추가합니다. 다음 실행의 컨텍스트로 주입되는 것이 이 요약본입니다.

6 요청의 성격에 맞는 응답

GISclaw는 받은 메시지가 어떤 종류인지 먼저 판단한 뒤 행동합니다. 덕분에 실행할 내용을 이미 아는 순간뿐 아니라 작업의 전 과정에서 사용할 수 있습니다.

- **분석 요청** — 실행을 시작합니다.
- **프로젝트에 대한 질문** — 무엇을 했는지, 어떤 레이어에 무엇이 들어 있는지, 어떤 값이 왜 이상해 보이는지 — 기록을 근거로 한 번의 호출로 답합니다.
- **검토하고 싶은 구상** — 함께 논의합니다. 접근 방식을 제시하고, 현재 데이터로 무엇을 할지 묻고, 두 방법을 견주어 보고, 계산이 시작되기 전에 계획을 확정할 수 있습니다.
- **일상적인 대화** — 그대로 대화로 처리합니다.
- **범위를 벗어난 요청** — 정중히 거절하고 처리 가능한 범위를 안내합니다.

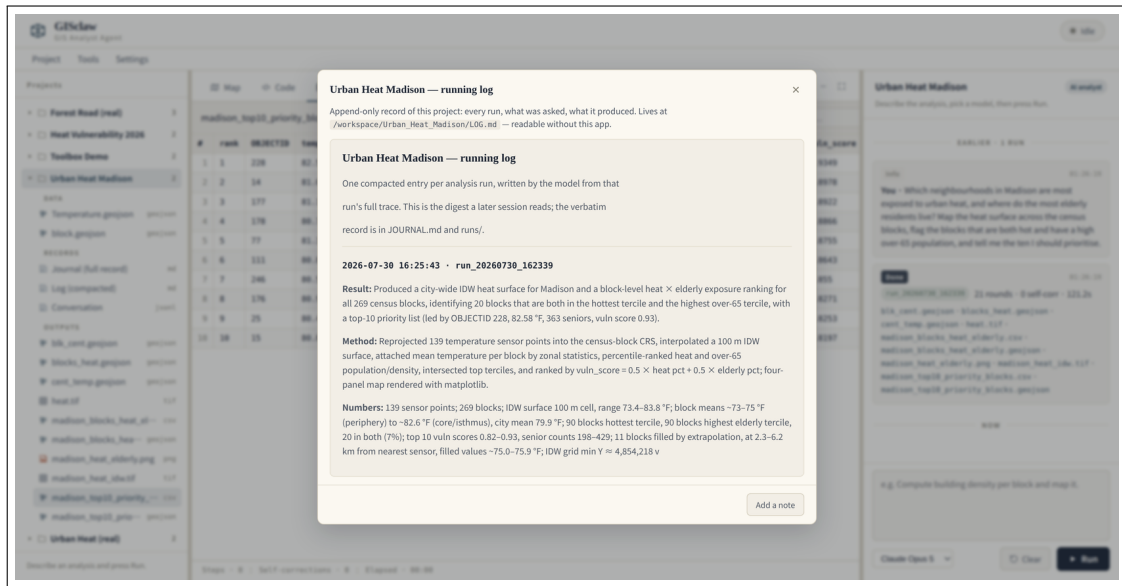


Figure 4: 압축 로그. 각 항목은 해당 실행의 전체 추적을 바탕으로 모델이 작성합니다.

방법을 논의하는 데는 짧은 호출 한 번이면 되지만, 실제로 실행하면 수 분과 1달러가 들 수 있습니다. 먼저 접근 방식에 합의하는 편이 올바른 답에 이르는 더 저렴한 경로인 경우가 많습니다.

7 상시 적용되는 선호

전역 MEMORY.md에는 모든 작업에 적용되는 규약을 둡니다. 색상 체계, 표준 분류 방식, 해당 지역의 투영 좌표계, 모든 산출 지도에 포함해야 할 요소 등입니다. **Settings** → **Memory...**에서 한 번 작성하거나 메시지 위에서 **Remember**를 누르면 되며, 모든 실행에 주입됩니다.

내용은 규칙 위주로 유지하십시오. HTML 주석 안의 글은 주입 전에 제거되므로 본인을 위한 메모는 비용이 들지 않습니다.

8 Skills

Skill은 특정 유형의 분석을 어떻게 수행할지 담은 디렉터리 번들입니다.

```
<skill>/
├── SKILL.md          YAML frontmatter와 짧은 라우터
├── manifest.yaml     선택적 선언형 로딩 계획
└── references/*.md   해당 단계에서 필요할 때만 열리는 상세 자료
```

로딩이 점진적이어서 라이브러리가 커져도 부담이 적습니다. 한 줄 설명만 상시 컨텍스트에 남고(약 80 토큰), 관련될 때 라우터가 로드되며, 참조 파일은 하나씩 열립니다.

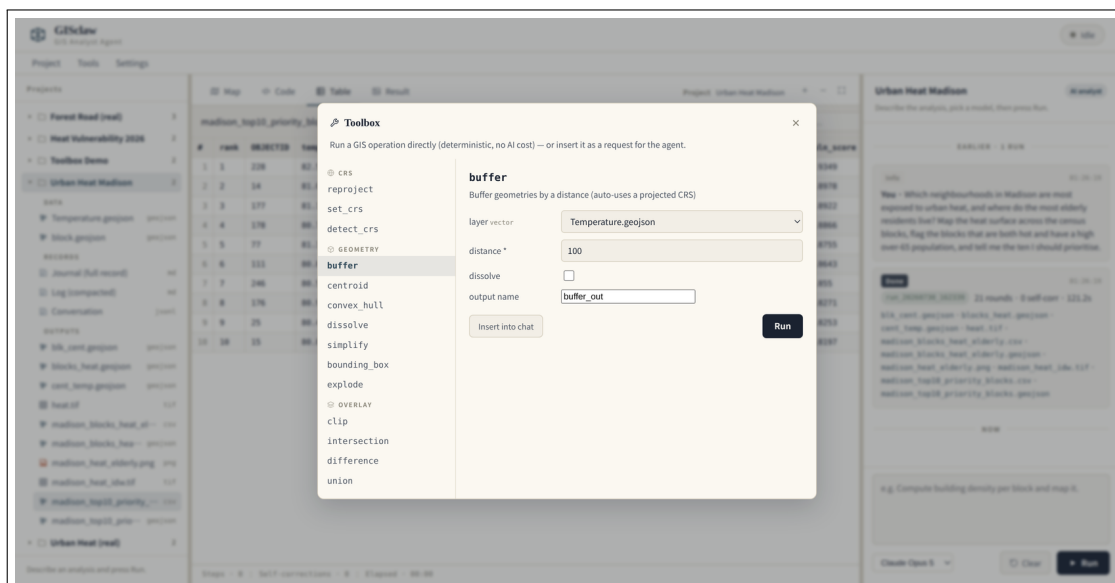
단계	내용	로드 시점
카탈로그	이름과 한 줄 설명	항상
라우터	SKILL.md 본문과 파일 목록	열거나 매칭될 때
상세	참조 파일 하나	해당 단계에서 필요할 때

기본 제공 스킬은 두 가지입니다. `gis-analysis-discipline`은 4장의 원칙을 담고 항상 적용되며, `gis-workflow-recipes`는 보간과 구역 통계, 다기준 적합도 중첩, 근접·영향 평가, 두 시점 변화 탐지의 단계별 템플릿을 제공합니다.

The screenshot shows the GitHub Copilot 'Settings' dialog box with the 'Skills' tab selected. The dialog is titled 'Settings' and has a close button (X) in the top right corner. Below the title bar, there are four tabs: 'API keys', 'Models', 'Skills', and 'Memory'. The 'Skills' tab is active, displaying a list of skills for the 'Urban Heat Map' project. The skills are categorized into two groups: 'gis-analysis-discipline' and 'gis-workflow-repices'. Each skill has a checkbox to enable it, a description, and a token count. The 'Enabled skills' section shows a list of skills with their respective token counts and a 'New' button. The 'Import' section shows a text input field for a folder path and an 'Import folder' button.

9 연산 도구 상자

두 가지 실용적 목적이 있습니다. 검증되어 있고 CRS를 올바르게 처리하므로 에이전트가 동등한 코드를 직접 작성하기 전에 먼저 사용합니다. 그리고 필요한 연산을 이미 알고 있을 때 패널에서 직접 실행하면 API 호출 비용이 전혀 발생하지 않으며 결과가 즉시 지도에 표시됩니다.



8

10 데이터의 위치

분석은 컨테이너 안에서 원본 형식(Shapefile, GeoTIFF, GeoPackage, CSV, NetCDF)을 대상으로 수행됩니다. GeoJSON이나 PNG로의 변환은 브라우저 표시 계층에서만 일어납니다. 파일 자체는 사용자의 컴퓨터에 남습니다. 모델 제공자에게 전달되는 것은 프롬프트와 에이전트가 데이터에 대해 관찰한 내용입니다.

11 문제 해결

증상	확인 사항
모델 선택기에 No model configured 표시	키가 설정된 제공자가 없습니다. Settings → API keys 에서 Test .
실행 즉시 키 오류 발생	선택한 모델의 제공자에 키가 없습니다. ready 로 표시된 모델을 선택하십시오.
호스트에서 래스터 연산 실패	래스터 작업은 컨테이너 안에서 실행하십시오. 호스트의 PROJ 데이터베이스 손상은 rasterio.warp 에 영향을 주며 벡터 작업에는 영향이 없습니다.
갱신 후에도 화면이 이전 상태	브라우저를 강제 새로고침하고 콘솔의 app.js 버전을 확인하십시오.
outputs/ 에 중간 파일이 보임	각 geoprocess 단계가 pred_results/ 에 기록되며, 중간 산출물도 최종 결과와 함께 복사됩니다.

서버 로그: `docker logs -f gisclaw_app`. 실행별 로그: `projects/<프로젝트>/runs/<run_id>/run.log`.

12 연구 논문과의 관계

arXiv:2603.26845 에 기술된 시스템은 통제된 실험을 위한 벤치마크 하네스입니다. 본 데스크톱 애플리케이션은 그 에이전트 코어 위에서 재구축되었으며 이후 상당히 분화했습니다. 인터페이스에서 사용 가능한 연산 도구 상자, 저널과 압축 로그를 갖춘 프로젝트 영속성, 점진적 공개를 갖춘 스킴 번들, 전역 선호 메모리, 실시간 모델 탐색이 그 예입니다.

논문에 보고된 실험 결과는 고정된 커밋의 연구용 하네스에서 생성되었습니다. 재현 작업에는 해당 아티팩트를 사용하십시오.

13 결과에 의존하기 전에

GISclaw는 분석 경로와 코드를 스스로 계획하고 작성하며, 그 계획은 대규모 언어 모델에서 나옵니다. 틀릴 수 있고, 확신에 찬 어조로 틀릴 수도 있습니다. 4절의 운영 원칙은 약 1,800회의 통제 실행에서 도출된 것으로 전형적인 실패 — 전부 NULL이 되는 조인, 좌표계 불일치, 관측 범위를 벗어난 보간, 조용히 채워진 결측값 — 를 뚜렷하게 줄이지만 없애지는 못합니다.

실행할 때마다 수행된 코드와 전체 추적, 산출물이 디스크에 남는 것은 확인할 수 있게 하기 위해서입니다. 확인하십시오. 결과는 결론이 아니라 전문가 검토를 위한 초안으로 다루고, 잘못된 답이 피해로 이어지는 영역 — 홍수·재해 지도, 재난 대응, 기반시설 입지, 환경영향평가, 토지 소유, 공중보건 — 에서는 특히 신중해야 합니다. GISclaw는 연구 도구이며, 그 산출물은 측량, 공학, 환경, 법률, 도시계획 자문이 아닙니다.

저작자는 모델이 만들어 낸 결과와 그에 근거한 결정에 대해 어떠한 책임도 지지 않으며, 본 소프트웨어는 어떠한 형태의 보증도 없이 배포됩니다. 또한 분석은 로컬에서 수행되지만 추론은 그렇지 않습니다. 데이터에 대한 설명 — 파일명, 컬럼명, 좌표계, 범위, 요약 통계, 프로그램 출력에 나타나는 표본값 — 이 사용자가 설정한 모델 제공자에게 전송됩니다. 전문은 `DISCLAIMER.md` 에 있습니다.

14 라이선스와 출처 표시

Copyright © 2026 Han Jinzhen. GISclaw 는 GNU Affero General Public License 버전 3 이상으로 배포되는 자유 소프트웨어입니다. 상업적 이용을 포함하여 어떤 목적으로든 무료로 사용할 수 있습니다. 수정한 버전을 배포하거나 수정한 버전을 네트워크 서비스로 제공하는 경우, AGPL 제13조에 따라 해당 사용자에게 그 버전의 완전한 소스를 동일한 라이선스로 제공해야 합니다. 수정 없이 실행하거나 내부 용도로만 수정하는 경우에는 공개 의무가 발생하지 않습니다.

GISclaw 를 독점 제품에 내장하거나 비공개 호스팅 서비스로 운영하려는 경우, 저작권자로부터 별도의 상용 라이선스를 받을 수 있습니다. `COMMERCIAL-LICENSE.md` 를 참조하십시오. 관련 논문의 연구 코드는 `paper-v2` 브랜치에서 MIT 라이선스로 유지되어, 발표된 결과가 논문이 인용한 조건 그대로 재현 가능합니다.

포함된 예제 데이터셋은 Apache-2.0 으로 배포되는 GeoAnalystBench (Zhang et al., 2025) 에서 가져온 것이며 별도의 인용 요건이 있습니다. 서드파티 자료의 전체 목록은 `examples/urban-heat-madison/README.md` 와 `THIRD_PARTY_NOTICES.md` 를 참조하십시오.

A 처음부터 설치하기

이 부록은 GitHub도, Docker도, API 키 발급도 해 본 적이 없다고 가정합니다. 순서대로 따라 하면 아무 것도 없는 컴퓨터에서 GISclaw가 실행되는 상태까지 도달합니다. 처음에는 한 시간 정도를 잡아 두십시오. 대부분은 입력이 아니라 내려받기를 기다리는 시간입니다.

A.1 필요한 세 가지

Docker

GISclaw가 쓰는 지리공간 라이브러리 — GDAL, GEOS, PROJ, GeoPandas, rasterio — 는 제대로 설치하기 까다롭기로 유명하고, 버전이 어긋나면 좌표 처리가 조용히 망가집니다. Docker는 이 문제를 우회합니다. 모든 것이 이미 설치되어 동작하는 상자 하나를 내려받는 방식이며, 컴퓨터의 나머지 환경과 분리되어 있습니다.

GISclaw 파일

코드와 설정이 담긴 폴더이며 GitHub에 공개되어 있습니다. GitHub는 그 폴더가 놓인 장소일 뿐입니다. 계정도 필요 없고, 버전 관리를 알아야 내려받을 수 있는 것도 아닙니다.

API 키

지리공간 계산은 GISclaw가 직접 수행하지만, 추론은 제공자의 서버에서 돌아가는 대규모 언어 모델이 담당합니다. 키는 그 제공자에서의 계정을 식별하고 사용량을 청구하기 위한 긴 비밀 문자열입니다. GISclaw 자체에는 계정도, 구독도, 라이선스 비용도 없습니다.

그 밖에는 아무것도 필요하지 않습니다. QGIS도, ArcGIS도, 파이썬 설치도 필요 없습니다.

A.2 1단계 --- Docker 설치

Windows 10 또는 11

1. <https://www.docker.com/products/docker-desktop/> 에서 **Docker Desktop**을 내려받되 CPU에 맞는 빌드를 고릅니다(거의 항상 **AMD64**이며, Snapdragon 기반 기기만 ARM64).
2. 설치 프로그램을 실행하고 **Use WSL 2 instead of Hyper-V** 를 체크된 상태로 둡니다.
3. 재시작하라는 안내가 나오면 재시작합니다.
4. Docker Desktop을 실행하고 고래 아이콘의 움직임이 멎어 Engine running 으로 바뀔 때까지 기다립니다.

가장 흔한 걸림돌은 BIOS/UEFI에서 하드웨어 가상화가 꺼져 있는 경우입니다. Docker Desktop이 명시적으로 알려 주며, 해당 항목은 메인보드에 따라 Intel VT-x, AMD-V, SVM Mode 등으로 표기됩니다.

macOS

같은 주소에서 내려받되 칩에 맞는 빌드를 반드시 고르십시오. M 시리즈는 **Apple Silicon**, 구형은 **Intel**입니다. 잘못 고르면 실행되지 않습니다. 확실하지 않다면 **Apple 메뉴 → 이 Mac에 관하여** 에서 확인하십시오. 응용 프로그램 폴더로 끌어다 놓고 실행한 뒤, macOS가 물어보면 권한 도우미 설치를 허용합니다.

Linux(Ubuntu, Debian 계열)

대부분의 배포판이 제공하는 docker.io 패키지는 여기서 쓰는 docker compose 문법을 지원하기에는 너무 낡았습니다. <https://docs.docker.com/engine/install/> 를 따라 Docker 공식 저장소에서 설치한 뒤, 매번 sudo 를 쓰지 않도록 사용자를 docker 그룹에 추가하십시오.

```
sudo usermod -aG docker $USER
```

로그아웃 후 다시 로그인해야 적용됩니다.

제대로 설치됐는지 확인

터미널에서 다음 세 가지를 실행합니다.

```
docker --version
docker compose version
docker run --rm hello-world
```

세 번째 명령은 아주 작은 테스트 이미지를 내려받아 Hello from Docker! 를 출력합니다. 출력된다면 설치의 정상이며, 이 부록의 나머지도 문제없이 진행됩니다.

하이픈이 들어간 docker-compose 는 구버전인 v1입니다. 본 설명서는 띄어쓰기 형태인 docker compose 를 사용합니다. 기기에 하이픈 형태만 있다면 계속하기 전에 업그레이드하십시오. v1은 이 프로젝트의 설정을 해석하지 못합니다.

다음에 입력하게 될 명령

Docker가 준비되면 나머지는 명령 세 줄과 브라우저 탭 하나입니다. 2단계부터 4단계까지 각 줄을 설명하고 잘 되지 않을 때의 대처도 다루지만, 전체 모습은 이렇습니다.

```
git clone https://github.com/geumjin99/GISclaw.git
cd GISclaw
docker compose up -d
```

그다음 브라우저에서 <http://localhost:8765> 를 열고 **Settings** → **API keys...** 에서 API 키를 붙여 넣습니다. 설치의 이것으로 끝입니다.

A.3 2단계 --- GISclaw 파일 받기

아래 두 방법 모두 가능하며, GitHub 계정은 어느 쪽도 필요하지 않습니다.

git 없이 받기(가장 간단)

1. <https://github.com/geumjin99/GISclaw> 에 접속합니다.
2. 초록색 **Code** 버튼을 누르고 **Download ZIP** 을 선택합니다.
3. 다시 찾기 쉬운 위치에 압축을 풉니다. 문서 폴더면 충분합니다. GISclaw-main 이라는 폴더가 생깁니다.

공백이나 한글이 없는 경로를 권합니다. 대개는 둘 다 문제없지만, 나중에 이상 동작이 생기면 가장 먼저 배제해야 할 요인입니다.

git으로 받기

```
git clone https://github.com/geumjin99/GISclaw.git
cd GISclaw
```

이후 git pull 만으로 전체를 다시 받지 않고 갱신할 수 있다는 장점이 있습니다.

그 폴더에서 터미널 열기

이후의 모든 명령은 docker-compose.yml 이 들어 있는 폴더 안에서 실행해야 합니다.

- **Windows** — 파일 탐색기에서 해당 폴더를 열고 주소 표시줄을 클릭한 뒤 cmd 를 입력하고 Enter. 또는 폴더 빈 공간에서 마우스 오른쪽 버튼을 눌러 **터미널에서 열기**.
- **macOS** — 폴더에서 마우스 오른쪽 버튼을 눌러 **서비스** → **폴더에서 새로운 터미널 열기**. 항목이 없으면 **시스템 설정** → **키보드** → **키보드 단축키** → **서비스** 에서 활성화합니다. 또는 터미널에 cd 를 입력하고 폴더를 창으로 끌어다 놓아도 됩니다.
- **Linux** — 폴더 안에서 마우스 오른쪽 버튼을 눌러 **터미널에서 열기**.

ls(macOS/Linux) 또는 dir(Windows)로 목록을 확인해 docker-compose.yml 이 보이면 위치가 맞습니다.

A.4 3단계 --- API 키 발급

사용료는 모델 제공자에게 직접 지불합니다. 비용은 크지 않습니다. 분석 한 번은 대체로 1센트 미만에서 수십 센트 사이이며, 어떤 모델을 쓰는지와 작업이 얼마나 오래 도는지에 따라 달라집니다.

아래 중 하나만 있으면 시작할 수 있습니다.

- **DeepSeek** — <https://platform.deepseek.com>
압도적으로 저렴하며 소프트웨어를 익히기에 충분합니다. 첫 선택으로 권합니다.
- **Anthropic(Claude)** — <https://console.anthropic.com/settings/keys>
Claude Opus 5는 GISclaw의 기본 모델이며, 여러 단계에 걸친 긴 분석에 가장 강합니다.
- **OpenAI** — <https://platform.openai.com/api-keys>
사용자가 많고 문서가 충실합니다.
- **Google Gemini** — <https://aistudio.google.com/apikey>
무료 사용량이 있어 비용 없이 처음 한 번 시도해 보기에 좋습니다.

절차는 어디서나 같습니다.

1. 위 주소의 콘솔에서 계정을 만듭니다.
2. 결제 수단을 등록합니다. 대부분은 계정에 잔액이 있어야 첫 호출이 성공하며, 일부는 소액의 체험 크레딧을 줍니다.
3. **API keys** 또는 **Credentials** 항목을 찾습니다.
4. 새 키를 만들고 GISclaw 처럼 알아보기 쉬운 이름을 붙입니다.

5. **즉시 복사하십시오.** 전체 키는 단 한 번만 표시됩니다. 잃어버렸다면 그 키를 삭제하고 새로 만드는 수밖에 없습니다. 다시 확인할 방법은 없습니다.

키는 신용카드처럼 다루십시오. 가진 사람은 누구든 잔액을 쓸 수 있습니다. 채팅 메시지, 스크린샷, 버그 리포트, 저장소에 커밋하는 파일에 절대 붙여 넣지 마십시오. 노출되면 즉시 제공자 콘솔에서 폐기하고 새로 발급하십시오. GISclaw는 키를 서버 측 `./projects/.gisclaw/settings.json`에 파일 권한 600으로 보관하며, 브라우저에는 `sk-abc...wxyz` 같은 마스크만 돌려줍니다.

제공자 콘솔은 자주 개편됩니다. 위에 적힌 위치가 다르다면 API keys, Credentials, Billing 같은 표현을 찾아보십시오.

A.5 4단계 --- GISclaw 실행

2단계에서 연 터미널에서:

```
| docker compose up -d
```

첫 실행은 이미지를 직접 빌드하므로 오래 걸립니다. Docker가 conda-forge에서 지리공간 스택을 받아 파이썬 패키지를 설치하며, 최종 크기는 약 1.8 GB입니다. 보통 회선에서 10분에서 30분 정도 걸리고, 중간에 아무 일도 일어나지 않는 것처럼 보이는 구간이 깁니다. 정상이며 내려받는 중입니다. 이후의 실행은 빌드된 이미지를 재사용하므로 몇 초면 끝납니다.

명령이 끝나면 브라우저에서 <http://localhost:8765>를 엽니다.

Settings → **API keys...**로 가서 3단계에서 받은 키를 해당 제공자 칸에 붙여 넣고 **Test**를 누릅니다. 테스트를 통과해야 그 제공자의 모델이 실행 선택기에 나타납니다. 모델 목록이 비어 보이는 이유도 이것입니다. 작동하는 키가 연결된 모델만 제공됩니다.

A.6 5단계 --- 첫 분석

예제가 함께 제공됩니다. 애플리케이션은 마운트된 작업 공간, 즉 `docker-compose.yml` 옆의 `projects` 폴더 안에 있는 파일만 볼 수 있으므로 예제를 먼저 복사합니다.

```
| cp -r examples/urban-heat-madison projects/
```

Windows에서는 같은 폴더에서:

```
| xcopy /E /I examples\urban-heat-madison projects\urban-heat-madison
```

그다음 브라우저에서 **Project** → **New project...**로 프로젝트를 만들고 이름을 붙인 뒤, **Project** → **Add data...**에서 GeoJSON 두 개를 선택하고, 인구조사 블록별 여름 폭염 노출 지도를 만들고 어느 블록이 가장 심한지 알려 줘 처럼 요청을 입력합니다. 오른쪽에서 작업 과정이 실시간으로 흘러갑니다.

A.7 자주 쓰게 될 명령

모두 `docker-compose.yml`이 있는 폴더에서 실행합니다.

명령	하는 일
<code>docker compose up -d</code>	백그라운드로 GISclaw 시작
<code>docker compose stop</code>	중지하되 컨테이너와 이미지는 유지
<code>docker compose down</code>	중지 후 컨테이너 삭제. projects/ 폴더는 그대로
<code>docker compose logs -f</code>	서버 로그 실시간 확인. Ctrl+C로 보기 종료
<code>docker compose restart</code>	재시작. 설정 파일을 바꾼 뒤에 사용
<code>docker compose up -d --build</code>	소스를 갱신한 뒤 이미지 재빌드

작업물은 컨테이너 안에 저장되지 않습니다. 프로젝트, 데이터, 산출물, 로그, 키 모두 사용자 디스크의 projects 폴더에 있으므로 컨테이너를 지우고 다시 만들어도 잃는 것이 없습니다.

A.8 작업물이 저장되는 곳과 이후 업데이트

만들어 낸 모든 것 — 프로젝트, 첨부한 데이터, 산출물, 실행 기록, 입력한 API 키 — 은 사용자 디스크의 한 폴더에 있습니다. 기본값은 압축을 푼 폴더 안의 projects 입니다. 컨테이너는 자체적으로 아무것도 보관하지 않으므로, 컨테이너를 지우고 다시 만들어도 작업물은 그대로 남습니다.

다만 이 기본값에는 날카로운 모서리가 있습니다. 압축을 푼 그 폴더는 새 버전이 나왔을 때 교체하게 되는 바로 그 폴더입니다. 새 릴리스를 그 위에 덮어쓰거나 옛 폴더를 지우면 프로젝트와 저장된 API 키가 함께 사라집니다. 게다가 Docker가 그 자리에 빈 폴더를 조용히 다시 만들기 때문에, GISclaw 는 아무 문제 없어 보이는 상태로 실행됩니다. 안이 비어 있을 뿐입니다.

한 줄만 설정하면 이 문제는 사라집니다. `docker-compose.yml` 옆에 `.env` 라는 파일을 만들고 소스 폴더 바깥의 경로를 적으십시오.

```
| GISCLAW_DATA=/home/you/GISclaw-data
```

기존 projects 폴더를 먼저 그곳으로 옮긴 뒤 `docker compose up -d` 를 실행합니다. 그 이후로 소스 폴더에는 사용자의 것이 남지 않으므로 자유롭게 지우거나 교체할 수 있습니다.

새 버전으로 업데이트

`GISCLAW_DATA` 를 설정했다면 소스 폴더를 교체하고 다시 빌드하면 됩니다.

```
| docker compose up -d --build
```

설정하지 않았다면 작업물을 먼저 옮겨야 합니다. 순서가 중요합니다.

1. `mv GISclaw/projects ~/gisclaw-projects`
2. 새 릴리스를 내려받아 압축을 풀고 옛 폴더를 지웁니다.
3. `mv ~/gisclaw-projects GISclaw/projects`
4. `docker compose up -d --build`

`--build` 는 생략할 수 없습니다. 없으면 Docker가 기존 이미지를 그대로 재사용해 새 코드가 컨테이너에 반영되지 않습니다.

2단계에서 `git`을 사용했다면 `git pull` 후 `docker compose up -d --build` 로 같은 일을 할 수 있고, 데이터 폴더는 전혀 건드리지 않습니다.

A.9 설치가 잘 되지 않을 때

아래 표는 실행 단계의 문제를 다룹니다. 분석 도중의 문제는 앞의 "문제 해결" 절을 참고하십시오.

증상	대처
docker: command not found	Docker가 설치되지 않았거나 PATH에 없습니다. 다시 설치하고, Windows와 macOS에서는 Docker Desktop이 실제로 실행 중인지 확인하십시오.
Cannot connect to the Docker daemon	엔진이 실행 중이 아닙니다. Docker Desktop을 켜거나 Linux에서 <code>sudo systemctl start docker</code> .
docker.sock 의 permission denied(Linux)	사용자가 docker 그룹에 없습니다. <code>sudo usermod -aG docker \$USER</code> 실행 후 로그아웃했다가 다시 로그인하십시오.
env file .env not found	Docker Compose가 2.24보다 낮은 버전입니다. 업그레이드하거나, 같은 폴더에 빈 .env 파일을 만드십시오.
port is already allocated	8765 포트를 다른 프로그램이 쓰고 있습니다. 그 프로그램을 끄거나 docker-compose.yml 을 "8766:8765" 로 고쳐 <code>http://localhost:8766</code> 을 사용하십시오.
내려받는 도중 빌드 실패	대개 네트워크 문제입니다. <code>docker compose up -d</code> 를 다시 실행하십시오. 완료된 레이어는 캐시되므로 처음부터 다시 받지 않습니다.
페이지가 열리지 않음	컨테이너가 종료됐을 수 있습니다. <code>docker compose logs</code> 로 마지막 줄을 확인하십시오.
모델 목록이 비어 있음	아직 Test 를 통과한 키가 없습니다. 3단계로 돌아가십시오.
화면이 예전 버전처럼 보임	강제 새로 고침: Ctrl+Shift+R, macOS에서는 Cmd+Shift+R.

A.10 비용을 낮게 유지하려면

- 익히는 동안에는 DeepSeek으로 시작하고, 결과가 중요해지면 더 강한 모델로 바꾸십시오.
- 잘 정리된 요청 하나가 모호한 요청 여러 개보다 저렴합니다. 의도를 파악하는 데 드는 라운드가 줄기 때문입니다.
- 이미 있는 것을 재사용하십시오. 앞서 만든 레이어에 열을 추가하는 편이 다시 계산하는 것보다 훨씬 쌉니다. GISclaw는 이전 산출물을 안내받으므로 그렇게 동작할 수 있습니다.
- 제공자 콘솔에서 월 지출 한도를 설정하십시오. 모든 제공자가 제공하며, 예상치 못한 과금을 확실히 막는 유일한 수단입니다.
- 실행마다 비용이 기록됩니다. **Project** → **Log (compact)** 와 **Journal** 에서 확인할 수 있습니다.